Trainer Guide: Spring Boot Session 2 - Hands-on Lab

A walk-through script for springboot-session2-lab.html - building the Student CRUD API endpoint by endpoint. Explain every step out loud as you lead.

Presenter: Purvam Prajapati - Faculty, AHD
Companion to: springboot-session2-lab.html

Each lab has an ON SCREEN reminder and a SAY / WALK THROUGH section you can read out as you lead. EMPHASISE and DELIVERY TIP notes are for you only.

Setup

Lab flow & progress tracker

ON SCREEN

The lab page with a hero header, sticky progress bar, an 8-step checklist, and expandable lab sections with copy buttons.

SAY / WALK THROUGH

Today we build a complete Student CRUD API, one endpoint at a time. The rhythm is: write a method, run it, test it, then move to the next. The progress bar at the top fills across eight milestones as we go. We keep one shared list of students as a field on the controller, so every endpoint - GET, POST, PUT, DELETE - works on the same data. The data lives in memory for now; it resets each restart. Making it survive restarts is exactly what Session 3 adds with a database.

Pre-requisites & project setup with Swagger

ON SCREEN

Pre-requisites checklist, then steps to create or reuse a project, add the springdoc Swagger dependency to pom.xml, and open the empty Swagger UI.

SAY / WALK THROUGH

Pre-requisites first: you can reuse the Session 1 springboot-demo project, or create a fresh one - either works. You'll want Postman, or you can use Swagger in the browser. And keep DevTools on so the app auto-restarts when you save. Now setup. Add the springdoc dependency to pom dot xml - copy it exactly, because the artifact ID is easy to mistype: springdoc-openapi-starter-webmvc-ui. Run the app and open slash swagger-ui slash index dot html. You'll see a page with no operations yet - that's expected. As we add controller methods, refresh this page and they appear automatically.

DELIVERY TIP

Stress DevTools: today we edit the controller many times. With spring-boot-devtools on the classpath the app auto-restarts on save, so you avoid stop-run between every endpoint. It keeps the session fast.

The labs - build the API

Lab 1 - Create the Student Bean

ON SCREEN

Steps to create a model package and a Student class with private fields, a no-arg and a parameterized constructor, and getters/setters.

SAY / WALK THROUGH

Lab one - the data model. Create a package com dot tcs dot training dot model, and a class Student with four fields: id, firstName, lastName, email. Add a no-argument constructor - Jackson needs that to build objects from incoming JSON - and a parameterized one for convenient test data. Then getters and setters. Why write getters and setters by hand when Lombok could generate them? Because today I want you to see exactly what they do: Jackson calls getFirstName to produce the firstName JSON key, and calls setFirstName to read it back in. The names matter. There's nothing to run yet - the controller in Lab 2 uses this class.

Lab 2 - Controller + Your First GET Endpoint

ON SCREEN

Steps to create a controller with @RestController and class-level @RequestMapping('/students'), and a @GetMapping returning one hardcoded Student. Expected JSON shown.

SAY / WALK THROUGH

Lab two - the controller and our first GET. Create a controller package and StudentController. Put at-RestController on the class, and at-RequestMapping slash students at the class level - that means every method's URL starts with slash students, so we don't repeat it. Add a method with at-GetMapping that returns a single hardcoded Student. Run it and open localhost slash students. You get JSON back - notice you returned a Java Student object and the client received JSON, with no conversion code from you. That's Jackson serializing automatically. Then open Swagger and confirm the endpoint now appears in the list - it built itself from your annotations.

Lab 3 - GET All Students (a JSON array)

ON SCREEN

Steps to add a field-level list of three students and a @GetMapping('/all') returning the list. Expected JSON array.

SAY / WALK THROUGH

Lab three - return many students. Add a field to the controller - an ArrayList of Student pre-filled with three records. We keep it as a field so the later POST, PUT, and DELETE labs all modify the same data. Then add a method, at-GetMapping slash all, that simply returns the list. Run and open slash students slash all. This time you get a JSON array - square brackets wrapping three objects. Again, you returned a Java List and the client received a JSON array, with zero conversion code. That's the auto-magic of at-RestController and Jackson, and it's the foundation everything else builds on.

Lab 4 - GET by ID with @PathVariable

ON SCREEN

Steps to add @GetMapping('/{id}') with @PathVariable int id, looping the list to find a match. Expected output for /students/2.

SAY / WALK THROUGH

Lab four - fetch one student by id. Add a method mapped to slash brace-id-brace, taking at-PathVariable int id. Inside, loop the list and return the student whose id matches. Here's the concept: at-PathVariable pulls a value straight out of the URL path. In slash students slash id, the id placeholder binds to your method's id parameter - so a request to slash students slash 2 calls the method with id equal to 2. Test slash students slash 1 and slash students slash 2 and watch the right record come back. For now, if no student matches, we return null - which isn't ideal. Note that out loud: in Session 3 we'll return a proper 404 instead.

Lab 5 - Search with @RequestParam

ON SCREEN

Steps to add @GetMapping('/search') with @RequestParam String name, filtering the list. Expected output for ?name=Ravi.

SAY / WALK THROUGH

Lab five introduces the other way to read input from a URL - the query parameter. Add a method mapped to slash search, taking at-RequestParam String name. Inside, filter the list for students whose first name matches. Test slash students slash search question-mark name equals Ravi. So when do you use which? Use at-PathVariable to identify a specific resource - it's part of the resource's identity, like slash students slash 1. Use at-RequestParam for filtering, searching, or optional criteria, like question-mark name equals Ravi. The rule of thumb: identity in the path, filters in the query.

Lab 6 - Create with @RequestBody (201 Created)

ON SCREEN

Steps to add a @PostMapping with @RequestBody Student and @ResponseStatus(CREATED), then test in Postman with a JSON body.

SAY / WALK THROUGH

Lab six - our first write operation. Add a method with at-PostMapping, taking at-RequestBody Student, which adds the student to the list. Put at-ResponseStatus of HttpStatus dot CREATED on it so it returns 201 instead of the default 200. At-RequestBody is the mirror of what we've seen - it takes the incoming JSON in the request body and deserializes it into a Java Student, via Jackson. You can't easily POST from a browser, so test in Postman: method POST, the slash students URL, set Content-Type to application slash json, and paste the JSON body from the sheet. You should get 201 Created back, and calling GET slash students slash all again shows the new student in the list.

DELIVERY TIP

Show it in Swagger too if time allows - expand the POST operation, click Try it out, paste the JSON, and Execute. It proves you don't even need Postman for a quick test.

Lab 7 - Update & Delete by ID

ON SCREEN

Steps to add @PutMapping('/{id}') combining @PathVariable and @RequestBody, and @DeleteMapping('/{id}'). Postman tests for each.

SAY / WALK THROUGH

Lab seven completes CRUD with update and delete. PUT combines both ideas we've learned: at-PutMapping slash id uses at-PathVariable to say which student, and at-RequestBody to carry the new data. Inside, find the student by id, replace its fields, and keep the id from the URL. Test in Postman with PUT to slash students slash 2 and a JSON body. DELETE is simpler: at-DeleteMapping slash id, take the at-PathVariable id, and remove that student from the list. Test DELETE to slash students slash 3, then GET slash students slash all to confirm it's gone.

EMPHASISE

Close the loop honestly: remind them these changes only persist while the app runs - restart it and the edits vanish, because the data lives in an in-memory list. That limitation is precisely what motivates the database in Session 3.

Bonus - ResponseEntity

ON SCREEN

A bonus lab refactoring GET-by-id to return ResponseEntity - 200 with the student, or a clean 404 when missing.

SAY / WALK THROUGH

If anyone finishes early, here's a bonus that previews Session 3's exception handling. Refactor GET-by-id to return a ResponseEntity instead of a raw Student. When the student exists, return ResponseEntity dot ok with it - that's a 200. When it doesn't, return ResponseEntity dot notFound dot build - a clean 404 with no body. Test slash students slash entity slash 1 for a 200, and slash entity slash 999 for a 404. That's far cleaner than the null we returned earlier - and it's a taste of the precise error handling we make standard next session.

Reference & close

Complete controller reference & wrap-up

ON SCREEN

The 'Complete StudentController.java' reference block and the annotation quick-reference card at the bottom.

SAY / WALK THROUGH

The bottom of the sheet has the entire StudentController as it looks after all the labs - the safety net. If anyone fell behind, they paste that, run it, and they're caught up with the group. There's also a quick-reference card listing every annotation we used today with a one-line description. To wrap: today you built a complete CRUD REST API - create, read, update, delete - using path variables, query parameters, request bodies, and proper status codes, and you tested it all in Postman and Swagger. In Session 3 we make it production-shaped: layers, DTOs, real exception handling, and a database. Excellent work.